Visitor

Visitor Pattern

Add operations without touching classes

DEFINITION

Represent an operation on elements of an object structure, letting you add new operations without changing the elements.

Visitor moves an operation out of an object structure into a separate visitor object with one method per element type. Each element exposes accept(visitor) and calls back the matching visit method — two dispatches (on element type, then operation) that together are double dispatch. A new operation is a new visitor; the elements never change.

WHY IT MATTERS

When many unrelated operations (type-check, pretty-print, evaluate) must run over a stable structure of node types, putting each as a method on every node smears it across classes and bloats them. Visitor gathers one operation's logic in one place and lets you add operations without editing the elements. The trade-off is the expression problem: adding a new element type now forces a new method in every visitor.

Vasyl Krupka · Ukraine

— How it works

Visitor	Declares visitConcreteA() / visitConcreteB() — one method per element type.
Element.accept(v)	The hook each element exposes; calls back v.visitX(this) — the second dispatch.
ConcreteVisitor	Implements one operation across all element types in one cohesive place.

HOW TO APPLY

1. Confirm the element types are stable but the operations keep growing.
2. Give each element an accept(v) that calls the matching v.visitType(this).
3. Put each operation in its own ConcreteVisitor with a method per type.
4. Walk the structure (Iterator / Composite) and accept a visitor at each node.

> LITMUS TEST
>
> Is the set of element types stable while operations keep multiplying? If you'd otherwise add the same method to every node class, lift it into a visitor.

Vasyl Krupka · Ukraine

— Smell → fix

Every node class carries a method for each operation — operations smeared across types:

```
class NumNode {
eval() { return this.v }
print() { return String(this.v) }
typeCheck() { /* ... */ } // add op → edit every node
}
```

↓ LIFT THE OPERATION INTO A VISITOR

```
interface Visitor { visitNum(n): number; visitAdd(a): number }
class Num {
v = 0
accept(x: Visitor) { return x.visitNum(this) } // dispatch
}
class Eval implements Visitor { // one op, all types
visitNum(n) { return n.v }
visitAdd(a) { return a.l + a.r }
}
```

Each node now just exposes accept(); Eval gathers the whole operation in one class. Add a Print or TypeCheck visitor without touching a single node — but a new node type means a new visit method in every visitor.

USE WHEN

- ✓ Operations over a stable AST/tree (type-check, pretty-print, eval)

AVOID WHEN

- ▲ Element types change often — Visitor makes that painful

— Trade-offs & pitfalls

PROS

+ Add operations easily
+ Group related logic together

CONS

− A new element type touches every visitor
− Double-dispatch overhead

COMMON PITFALLS

▲ The expression problem: easy to add operations, painful to add element types — every visitor must gain a method. Pattern matching flips the trade-off.

▲ Encapsulation leak: visitors usually need element internals, so accessors get exposed the element wouldn't otherwise publish.

▲ A cyclic Element-to-Visitor dependency and double-dispatch boilerplate; brittle while element types still churn.

WHERE YOU SEE IT

- Compiler / AST tooling: TS compiler and Babel visitors run checks, transforms, codegen; ESLint rules keyed by node type.
- .NET Roslyn CSharpSyntaxVisitor / SyntaxWalker; document exporters visiting nodes to emit HTML / PDF.
- Any type-check / pretty-print / evaluate pass over a stable tree.

SEE ALSO

Composite	Visitor usually runs over a Composite tree of elements
Iterator	walks the structure; Visitor performs a type-specific operation at each node
Interpreter	an AST is a natural Visitor target — add operations without bloating nodes
Strategy	differs by dispatching on element type, not just plugging one algorithm

Vasyl Krupka · Ukraine

Interview & sources

What is double dispatch?

Two virtual calls: element.accept(v) resolves the element's type, then v.visitX(this) resolves the operation. The method that runs depends on both types at once.

What's the expression problem here?

Visitor makes new operations cheap but new element types expensive — every visitor must gain a method. Pattern matching flips that trade-off.

Why not just an if/instanceof chain?

It works for a few types, but scatters each operation's type checks and forgets new types. Visitor centralizes an operation and (with a base) forces coverage.

Main downsides?

Boilerplate, a cyclic Element-to-Visitor dependency, broken element encapsulation, and brittleness when element types change.

REMEMBER

Pull operations out of the elements into a visitor — add new operations without touching the element classes.

SOURCES

GoF — "Design Patterns" (1994): Visitor (behavioral)

Wadler — "The Expression Problem" (1998): the operations-vs-types trade-off Visitor makes